

Functional Programming Exam Part 2 of 2

Course responsible

Examiner David Sands [mobile: 0737207663] will visit the exam rooms once at around 15–15:30 and again at 16:00-16.30, and is available by phone when not visiting the exam rooms. For students with extra time the visits will be around 30 minutes before these times.

Grading

There are two parts to this exam. This is Part 2.

You should have completed Part 1 before starting Part 2. Because of limitations of the SafeBrowser system, you cannot have two exams open at the same time.

Grading scheme for Part 2:

Part 2 consists of small programming assignments with parts requiring a short written response. There are three questions of $11 + 9 + 12 = 32$ points.

- Chalmers: 24 points or more are required for a 5, and 14 points for a 4.
- GU: 24 points are required for a VG

Any bonus points (maximum 4 points = 12.5%) will only be added if they make a difference your overall grade. In that case you should contact the course responsible.

Your score in Part 1 does not influence your score on Part 2.

Notes

- If you give multiple answers to the same question only the first answer will be graded.
- In your answers, you do not need to copy code or types which are already given in the question.
- For each part Question, if your solution consists of more than a few lines of Haskell code, use your common sense to decide whether to include a short comment to help your grader understand the solution.
- You can use any of the standard Haskell functions listed in the PDF accessible at the bottom of the SafeBrowser window.
- Full points are given to solutions which are short, elegant, and correct. Fewer points may be given to solutions which are unnecessarily complicated, unstructured, or repetitive.
- You are encouraged to use the solution to an earlier part of a Question to help solve a later part — even if you failed in solving the earlier part.

1 This question is about multi-way trees, sometimes called **rose trees**.

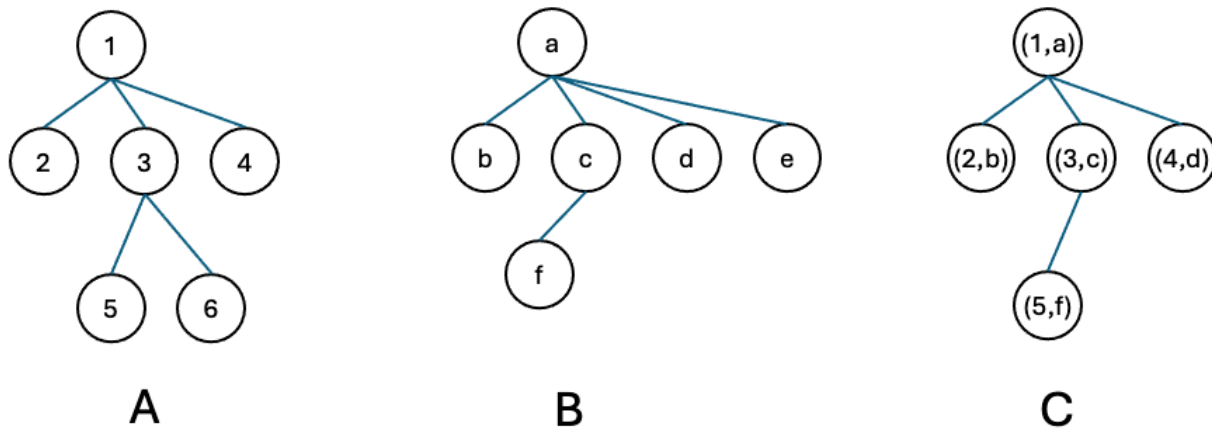
A rose tree is a tree with items at the nodes and where each node can have zero or more subtrees. This can be represented by the following data type in Haskell:

```
data Rose a = Node a [Rose a]
  deriving (Eq, Show)
```

(i) [3 points] The idea of zipping trees can be defined in a similar style to zip on lists. Define a function

```
zipRose :: Rose a -> Rose b -> Rose (a,b)
```

so that, for example, zipping the trees A and B gives the tree C pictured below:



(ii) [1 point] Let us say that a rose tree is "list-shaped" if every node matches either the pattern **Node x [t]** or **Node x []**. Define a function

```
fromList :: [a] -> Rose a
```

which converts any non-empty list into the corresponding list-shaped tree. It should be undefined on the empty list.

(iii) [3 points] Using **fromList**, define **prop_zipRose**, a property that checks that **zipRose** does essentially the same thing as **zip** when its inputs are list-shaped.

(iv) [4 points] Make **Rose** an instance of class **Functor**. (Hint: for **F** be a member of functor you need to define just one function, **fmap :: (a -> b) -> F a -> F b**)

Fill in your answers to parts (i)-(iv) here.

--	--

Maximum marks: 11

- 2 This question is about the QuickCheck function **listOf**, and a related function. In all parts of this question you may **not** use **vectorOf**, and you may **only** use functions explicitly listed in the given list of functions, as well as **arbitrary :: Gen Int**.

Define **listOf** in two ways:

- (i) [4 points] with the help of recursion, and
- (ii) [3 points] without recursion, using the given standard functions (except for **vectorOf**)

The final part-question is about another function for generating lists of things, this time with an extra argument:

- (iii) [2 points] Define a function

```
listContaining :: Gen a -> a -> Gen [a]
```

such that **listContaining g x** generates lists of elements of the same type as **x**, such that the list is guaranteed to contain at least one copy of **x**, and all other elements are generated by **g** (which may or may not generate other copies of **x**). The generated lists should be suitable for testing properties of lists which are required to contain a certain element at an arbitrary place in the list.

Fill in your answers here

1

Maximum marks: 9

- 3 This question is about finding the best place to add a lift (elevator) in a workplace based on optimising the number of steps that are taken by employees.

A workplace is on several floors. Each floor is separated by a certain number of steps. These things can be modelled by the following types:

```
type Steps = Int -- non-negative
type Floor = Int -- non-negative
type Workplace = [Steps] -- non-empty

example :: Workplace
example = [30,20,20,20,40]
```

This example workplace represents a building with 6 floors, with floors numbered 0 to 5. The number of steps between floor 0 (ground floor) and floor 1 is 30, whereas the number between floor 1 and 2 is 20, etc.

The activity of an employee can be modelled as a list of the floors they visit in a working day:

```
type Activity = [Floor]
```

For example, `[2,1,5,0]` represents the movements of Dave (in the example workplace above), who wakes up in his office on floor 2, visits the canteen on floor 1, the coffee room on floor 5, then leaves on floor 0.

- (i) [4 points] Define a function

```
stepCount :: Workplace -> Activity -> Steps
```

which computes the total number of steps in the given day's activity.

- (ii) [8 points] The workplace wants to install a lift (elevator). Unfortunately, they can only afford a lift that goes between two consecutive floors. To decide on which floor the lift should be installed on (we say that the lift is on floor n if it goes between floors n and $n+1$) they take the data from the last year (a list of the activities of all employees), and choose to install the lift on the floor that will minimise the number of steps that are taken - assuming that everyone will always take the lift when possible, even if they have to go from the stairs to the lift and then back again.

Define a function

```
bestPlace :: Workplace -> [Activity] -> Floor
```

which calculates the best place to install the lift. If there are several best choices then you may return any one of them. You may assume that the floor numbers are always valid for the given workplace, and that the workplace is non empty and has at least two floors.

Hint: the effect of putting the lift on floor n can be calculated by changing the n th element in the workplace to 0.

For maximum points you should break your solution down into meaningful parts which should be given suitable names and type signatures.

Fill in your answers to parts (i) and (ii) here

1	
---	--

Maximum marks: 12